Лабораторная работа 1 Репозиторий команды и воспроизводимая среда

Версия: 2026–2027 учебный год
Формат: командная работа по 2–5 человек с индивидуальной сдачей
Продолжение: результат ЛР1 используется во всех последующих лабораторных работах

1 Назначение работы

Команда должна подготовить единый GitHub-репозиторий учебного ML-проекта и доказать, что новый участник может получить код, создать изолированное окружение, установить зависимости и выполнить базовые проверки только по README. В ЛР1 оценивается не сложность модели, а надежность командного процесса и воспроизводимость стартового состояния проекта.

2 Результаты обучения

После выполнения работы каждый студент должен уметь:

- объяснить различия между рабочей копией, локальным и удаленным репозиторием;
- создать ветку от актуального main, связать изменение с issue и открыть pull request;
- выполнить содержательное review изменения другого участника;
- восстановить Python-окружение по зафиксированным требованиям;
- объяснить назначение README, CONTRIBUTING, CODEOWNERS, ADR и CI workflow;
- показать, почему секреты, персональные данные, модели и локальное окружение не должны попадать в Git.

3 Командный результат

К завершению ЛР1 команда предоставляет:

1. fork тимлида от ветки своей команды в репозитории преподавателя и отдельные forks участников от назначенных веток тимлида;
2. ветку main тимлида и ветки участников, защищенные от прямых изменений организационными или техническими правилами;
3. не менее одного закрытого issue, связанного с принятым pull request;
4. review от участника, который не является автором последнего изменения;
5. README с проверенными командами установки, тестирования и запуска;
6. CONTRIBUTING с правилами веток, commit, pull request и review;
7. CODEOWNERS после замены учебных заполнителей реальными GitHub-логинами;
8. ADR 0001 с границами проекта, пользователем, метрикой успеха и основными рисками;
9. успешный CI run для принятого pull request;
10. неизменяемую контрольную точку lr1-v1 в виде Git tag или GitHub Release.

4 Подготовка

На рабочей машине требуются Git, GitHub-аккаунт, Python 3.12 и редактор кода. Docker для ЛР1 необязателен. Используйте стартовый проект 2026-2027/starter-project или выданный преподавателем GitHub template repository.

Проверьте инструменты:

```
git --version
python --version
python -m pip --version
```

Настройте автора commit на уровне репозитория или пользователя. Указывайте собственные данные, связанные с GitHub-аккаунтом:

```
git config user.name "Имя Фамилия"
git config user.email "адрес из настроек GitHub"
```

Не публикуйте токены, пароли, .env, приватные ключи и реальные персональные данные. Для переменных окружения храните только безопасный .env.example без значений секретов.

5 Роли команды в ЛР1

Роль	Ответственность	Индивидуальное подтверждение
Release lead	Формирует план ЛР1, контролирует связи issue–PR и создает tag lr1-v1	Закрытый issue, merge или release notes
Repository administrator	Настраивает доступ, правила main, шаблоны issue и pull request	Снимок или экспорт правил и проверяемый PR
Environment engineer	Проверяет Python-версию, установку зависимостей и чистый запуск	Изменение README, .python-version или сценария установки
Quality engineer	Настраивает и проверяет lint, тесты и CI	Workflow run, тест или исправление проверки
Documentation and risk reviewer	Оформляет ADR, CONTRIBUTING, CODEOWNERS и проверяет риски	Документ, содержательное review или запись решения

В команде из 2–4 человек роли совмещаются. В команде из пяти каждому назначается отдельная зона результата. Координация без технического, документального или проверочного артефакта не считается индивидуальным вкладом.

6 Рекомендуемая структура репозитория

```
project/
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   └── lab-task.yml
│   ├── workflows/
│   │   └── ci.yml
│   └── pull_request_template.md
├── data/
├── docs/
│   ├── adr/
│   │   └── 0001-project-boundaries.md
│   ├── evidence/
│   │   └── lr1-evidence-card.md
│   └── team-roles.md
├── src/
├── tests/
├── .gitignore
├── .python-version
```

```
├── CODEOWNERS
├── CONTRIBUTING.md
├── pyproject.toml
└── README.md
```

Данные, модели и артефакты экспериментов в ЛР1 не добавляются в Git, кроме небольшого разрешенного учебного примера. Политика управления данными будет уточнена в ЛР2.

7 Порядок выполнения

7.1 Создание команды, forks и веток

Преподаватель ведет ветку каждой команды, например team-01, в репозитории курса. Тимлид делает fork репозитория преподавателя и работает в своем fork: его main — интеграционная ветка команды. В fork тимлида создается отдельная ветка на каждого участника, например student-01, student-02.

Каждый участник делает fork **своей назначенной ветки тимлида**, а не независимый новый шаблон. В своем fork участник работает в main и открывает PR в соответствующую ветку тимлида. Так история личного вклада и review сохраняются проверяемыми.

1. Преподаватель фиксирует состав команды из 2–5 человек и ветку команды в репозитории курса.
2. Тимлид делает fork, создает ветки участников и сообщает каждому назначение.
3. Каждый участник делает fork назначенной ветки тимлида, записывает роль в docs/team-roles.md и настраивает remotes.
4. Участник создает PR из student-fork:main в teamlead-fork:student-XX.
5. Тимлид после review принимает PR участников, затем открывает и принимает PR из веток участников в teamlead-fork:main.
6. Для ЛР1 тимлид открывает PR из teamlead-fork:main в teacher-repository:team-XX. Ссылка на этот PR прикрепляется в СКИФ.

Пример настройки локальной копии участника:

```
git clone https://github.com/STUDENT_LOGIN/TEAM_REPOSITORY.git
cd TEAM_REPOSITORY
git remote add teamlead https://github.com/TEAMLEAD_LOGIN/TEAM_REPOSITORY.git
git remote add teacher https://github.com/Irina-64/T26_MLOps.git
git remote -v
git status
```

Для ЛР2–ЛР15 в СКИФ прикрепляется ссылка на main fork-а тимлида после интеграции личных PR. Для ЛР16 тимлид вновь открывает PR из своего main в ветку команды преподавателя и прикрепляет его ссылку в СКИФ. Прямая отправка изменений в репозиторий преподавателя не допускается.

7.2 Воспроизводимое окружение

Создайте окружение в каталоге проекта и установите пакет с зависимостями для разработки.

Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

Linux или macOS:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

Выполните проверки:

```
python -m ruff check .
python -m pytest -q
```

Если команда использует другой менеджер окружений, она сохраняет эквивалентный lock-файл и добавляет проверенные команды в README. Локальное окружение .venv не коммитится.

7.3 Работа через issue и личный PR

Каждый участник создает или получает issue с наблюдаемым результатом и критериями приемки. Рабочая ветка создается от обновленного main **собственного fork-а**, затем изменения возвращаются PR в его назначенную ветку fork-а тимлида:

```
git switch main
git pull --ff-only
git switch -c docs/issue-12-reproducible-setup
```

После изменения проверьте diff и включайте в commit только относящиеся к задаче файлы:

```
git status
git diff
git add README.md .python-version
git commit -m "docs: document reproducible setup"
git push -u origin docs/issue-12-reproducible-setup
```

После push откройте PR STUDENT_LOGIN:docs/issue-12-reproducible-setup → TEAMLEAD_LOGIN:student-XX. Тимлид не подменяет личный PR прямым commit: сначала он одобряет PR участника, затем интегрирует принятые ветки в свой main. Не используйте git add ., если не проверили список файлов. В истории не должно быть секретов, локального окружения, кэшей, моделей и случайных наборов данных.

7.4 Pull request и review

Личный pull request должен ссылаться на issue, описывать результат и содержать способ проверки. Автор назначает reviewer из своей команды или тимлида. Reviewer:

- воспроизводит указанные команды либо проверяет связанный CI run;
- просматривает весь diff;
- оставляет конкретный комментарий, одобрение или запрос изменений;
- повторно проверяет изменение после исправлений.

Автор не утверждает собственный pull request. После новых commit требуется проверить, что одобрение относится к актуальному состоянию diff. Тимлид также не принимает собственный личный PR без независимого review.

7.5 Правила ветки main

Для main fork-а тимлида и для веток участников включите доступные в используемом плане GitHub правила:

- merge только через pull request;

- минимум одно одобрение другого участника;
- обязательное разрешение обсуждений;
- обязательная успешная проверка quality или эквивалентный CI job;
- запрет force push и удаления ветки;
- при возможности — одобрение последнего изменения участником, который его не отправлял.

Если учебный тариф или тип репозитория не позволяет применить правило технически, команда документирует эквивалентный процесс в CONTRIBUTING и демонстрирует его соблюдение по истории pull request. Ограничение платформы не отменяет review. До PR тимлида в ветку команды преподавателя в его main должны быть интегрированы и проверены личные PR всех участников.

7.6 Базовый CI

В стартовом проекте используется проверяемый workflow. На дату актуализации официальные репозитории действий рекомендуют актуальные major-версии actions/checkout@v7 и actions/setup-python@v7; перед новым потоком преподаватель сверяет их повторно.

```yaml
name: CI

on:
  push:
  pull_request:

permissions:
  contents: read

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
      - run: python -m pip install --upgrade pip
      - run: python -m pip install -e ".[dev]"
      - run: python -m ruff check .
      - run: python -m pytest -q
```

В ЛР1 не добавляйте секреты и публикацию образов. Разрешения workflow оставляйте минимальными.

7.7 ADR 0001 и границы проекта

В docs/adr/0001-project-boundaries.md зафиксируйте:

- учебную задачу и предполагаемого пользователя результата;
- входные данные без загрузки закрытого набора;
- основную метрику качества и эксплуатационный показатель;
- компоненты, которые войдут в итоговый проект;
- минимум три риска: данные, воспроизводимость и эксплуатация;

- решение, которое команда сознательно откладывает до следующих ЛР.

ADR описывает принятое решение и его последствия, а не пересказывает задание.

7.8 Контрольная точка

После принятия всех обязательных изменений:

```
git switch main
git pull --ff-only
git tag -a lr1-v1 -m "Laboratory 1 accepted baseline"
git push origin lr1-v1
```

Tag создается только на commit, прошедшем обязательный CI и review.

8 Сценарий приемки командного результата

Преподаватель или участник другой команды выполняет проверку в новом каталоге:

1. клонирует репозиторий по HTTPS;
2. создает чистое окружение по README;
3. устанавливает зависимости;
4. запускает Ruff и pytest;
5. находит закрытый issue, связанный pull request и review;
6. проверяет tag lr1-v1, ADR 0001 и отсутствие секретов;
7. сопоставляет правила main с CONTRIBUTING.

Командный результат не принимается, если успешный запуск зависит от незадокументированного локального файла, личного пути или устного пояснения автора.

9 Индивидуальная сдача участника команды

Каждый участник сдает ЛР1 отдельно и до защиты заполняет docs/evidence/lr1-evidence-card.md. Совпадающие карточки участников не принимаются.

Поле	Что указать для ЛР1
Студент	ФИО, учебная группа, команда и GitHub login
Контрольная версия	Ссылка на tag lr1-v1 и commit SHA
Роль	Роль в ЛР1 и зона ответственности
Задача	Собственная issue с критериями приемки
Личный артефакт	PR, содержательное review, workflow, README, CONTRIBUTING, CODEOWNERS, ADR или настройка правил
Вклад	Конкретное созданное, исправленное или проверенное изменение
Решение	Выбранный вариант и проверяемая причина выбора
Риск	Что могло пойти неверно и как это обнаруживается
Результат обучения	Что участник теперь может повторить самостоятельно
Использование ИИ	Инструмент, цель, способ проверки результата или отметка об отсутствии использования

9.1 Допуск к защите

Участник допускается, если:

1. командная версия зафиксирована tag lr1-v1;
2. личная issue содержит наблюдаемый результат;
3. есть содержательный личный артефакт в общей версии команды;
4. изменение прошло review другого участника либо сам участник выполнил содержательное review чужого изменения;
5. ссылки в evidence card открываются и соответствуют истории GitHub;
6. обязательный CI завершился успешно.

Количество commit само по себе не подтверждает вклад.

9.2 Индивидуальная защита

За 2–4 минуты участник:

1. показывает свой артефакт в командной версии;
2. объясняет входы, результат, критерий корректности и один риск;
3. выполняет одну операцию по выбору преподавателя: находит изменение через git log, объясняет diff, воспроизводит проверку, показывает связь issue–PR, интерпретирует CI log или объясняет правило main;
4. отвечает на один вопрос по соседнему компоненту, связанному с личным вкладом.

9.3 Стоп факторы

- чужой или отсутствующий личный артефакт;
- фиктивный commit либо искусственное дробление без самостоятельного результата;
- неработающая ссылка или несоответствие evidence card истории репозитория;
- секреты, персональные данные, локальное окружение или запрещенные данные в Git;
- прямой push в main в обход принятого процесса;
- неспособность объяснить собственное изменение.

9.4 Повторная сдача

Участник создает или переоткрывает issue с перечнем исправлений, вносит изменения отдельным pull request, получает новое review, обновляет evidence card и повторно защищает исправленную часть. История первой попытки не удаляется.

10 Оценивание

Критерий	Баллы	Проверка
Работоспособность	0–3	Репозиторий клонируется, окружение устанавливается, команды запуска выполняются
Воспроизводимость	0–2	Версии и зависимости зафиксированы, новый участник следует только README
Качество и безопасность	0–2	CI успешен, .gitignore корректен, секреты и лишние артефакты отсутствуют
Командный процесс	0–1	Issue, pull request и review прослеживаются, правила main соблюдены
Индивидуальная защита	0–2	Участник доказывает вклад и объясняет решение и риск

ЛР1 зачтена при результате не ниже 6 из 10 и отсутствии стоп-фактора. Командные пункты создают общий базовый балл, но итог каждого участника может отличаться по evidence card и защите.

11 Чек лист команды

- ☐ В репозитории работают ссылки из README.
- ☐ .venv, .env, кэши, модели и неразрешенные данные отсутствуют в Git.
- ☐ Все участники имеют собственные issue и проверяемые артефакты.
- ☐ Есть принятый pull request с review другого участника.
- ☐ CI выполняет Ruff и pytest с минимальными разрешениями.
- ☐ Настроены или документированы правила защиты main.
- ☐ Заполнены CONTRIBUTING, CODEOWNERS, ADR 0001 и таблица ролей.
- ☐ Evidence card каждого участника содержит уникальные рабочие ссылки.
- ☐ Tag lr1-v1 указывает на проверенный commit.

12 Вопросы для самопроверки

1. Чем git fetch отличается от git pull?
2. Почему ветку следует создавать от актуального main?
3. Что именно подтверждает pull request, а что — review?
4. Почему автор не должен быть единственным reviewer собственного изменения?
5. Какие файлы нельзя добавлять в Git и почему одного удаления недостаточно, если секрет уже попал в историю?
6. Чем фиксированная версия Python отличается от списка зависимостей проекта?
7. Зачем CI задавать минимальные permissions?
8. Как доказать, что tag lr1-v1 соответствует принятой версии?

13 Источники

- GitHub Docs: Protected branches и Pull request reviews.
- GitHub Docs Issue and pull request templates
- GitHub Actions: checkout и setup-python. Major-версии проверены 24 сентября 2026 года.